
Los Sabrossos

Contents

1	Math	1
1.1	Sieve of Eratosthenes	1
1.2	Euler Phi	1
1.3	Binary Exponentiation	1
1.4	Chinese Rest Theorem	1
1.5	Extended Euclides	2
1.6	Fibonacci Log(n)	3
1.7	Least Common Multiple	3
1.8	Modular Inverse	3
1.9	Pollard Rho Rabin Miller	3
2	Geometry	4
2.1	Convex Hull	4
2.2	Point	5
2.3	Closest Pair of Points Lineal	5
2.4	Closest Pair of Points Nlogn	6
3	Strings	7
3.1	KMP	7
3.2	Z Function	8
3.3	Trie	8
3.4	Suffix Array	9
3.5	Aho Corasick	10
3.6	Booths Algorithm	12
3.7	Hashing 2D	13
3.8	Hashing	14
3.9	Manacher	15
4	Bit Manipulation	15
4.1	Binary Representation	15
4.2	Bit Mask	15
4.3	Bits Prendidos En Un Numero X En Una Posicion Pos	15
4.4	Turn Off a Bit at Position X	16
4.5	Xor Basis	16
5	Data Structures	16
5.1	Segment Tree	16
5.2	Segment Tree Lazy Propagation	17
5.3	Bit 1D	18
5.4	Bit 2D	18
5.5	Bit 2D Infinite Plane	19
5.6	Bit 3D	20
5.7	DSU Dynamic Representative	21
5.8	Disjoint Set Union	21
5.9	Dynamic Connectivity Offline	21
5.10	Implicit Treap	23
5.11	Ordered Set	24
5.12	Range Bit	24
5.13	Sparse Table	25
5.14	Treap	25

6	Dynamic Programing	26
6.1	Bounded Knapsack	26
6.2	Kadane 2D	27
6.3	Kadane 3D	27
6.4	SOS DP	28
6.5	Fastfibo	28
7	Flow	29
7.1	Dinic	29
8	Graph Algorithms	30
8.1	Dijkstra	30
8.2	Kruskal MST	30
8.3	Topological Sort Kahn Algorithm	31
8.4	Kosaraju	31
8.5	Kuhn	32
8.6	2SAT	32
9	Tree Algorithms	33
9.1	Euler Tour	33
9.2	Binary Lifting	33
9.3	Centroid Decomposition	34
9.4	HLD	35
9.5	Lowest Common Ancestor	37
10	Utilities	37
10.1	Prime Numbers	37
10.2	ASCII TABLE	38
10.3	Int128	39
10.4	Merge Sort	39
10.5	Prefix Sum 2D	40
10.6	Priority Queue Min Heap	40
10.7	Random	40
10.8	Plantilla Python	40
10.9	Pragma	41

1. Math

1.1. Sieve of Eratosthenes

```
1  const int N = 1e7;
2  int sieve[N];
3  void make(){
4      for(int i = 0; i < N; i++) sieve[i] = 1;
5      sieve[0] = sieve[1] = 0;
6      for (int i = 2; i < N; i++)
7          {
8              if(sieve[i]){
9                  for (int j = i*i; j < N; j+=i)
10                     {
11                         sieve[j] = 0;
12                     }
13             }
14         }
15 }
```

1.2. Euler Phi

```
1  ll phi(ll n) {
2      ll ans = n;
3      for (ll p = 2; p * p <= n; p++) {
4          if (n % p == 0) {
5              ans -= ans / p;
6              while (n % p == 0){
7                  n /= p;
8              }
9          }
10     }
11     if (n > 1){
12         ans -= ans / n;
13     }
14     return ans;
15 }
```

1.3. Binary Exponentiation

```
1  long long pote(long long a, long long b, long long mod){
2      long long ans = 1ll;
3      while(b){
4          if(b&1) ans = (ans * a)%mod;
5          a = (a * a)%mod;
6          b>>=1;
7      }
8      return ans;
9  }
10 // a^-1 = a^mod-2 = pote(a,mod-2);
```

1.4. Chinese Rest Theorem

```
1  //si los modulos son muy grandes 1e18, cambiar por int128 las operaciones
2  ll extgcd(ll a, ll b, ll &x, ll &y) {
3      if (!b) {
4          x = 1;
5          y = 0;
6          return a;
7      }
8
9      ll x1, y1;
10     ll g = extgcd(b, a % b, x1, y1);
11
12     x = y1;
13     y = x1 - (a / b) * y1;
14
15     return g;
16 }
17
18 // x      a (mod m)
19 // x      b (mod n)
20 //si retorna {0, -1} -> no solution
21 pair<ll,ll> CRT(ll a, ll m, ll b, ll n) {
22     ll x, y;
```

```

23     ll g = extgcd(m, n, x, y);
24
25     if ((b - a) % g != 0)
26         return {0, -1};
27
28     ll n2 = n / g;
29
30     ll t = ((b - a) / g * x) % n2;
31     if (t < 0) t += n2;
32
33     ll mod = m * n2;
34     ll r = (a + m * t) % mod;
35     if (r < 0) r += mod;
36
37     return {r, mod};
38 }
39
40 // x      a[i] (mod m[i]) para todo i
41 //vector de valores y vector de respectivos modulos
42 pair<ll,ll> CRT(vector<ll> a, vector<ll> m) {
43     ll r = a[0];
44     ll mod = m[0];
45
46     for (int i = 1; i < a.size(); i++) {
47         pair<ll,ll> res = CRT(r, mod, a[i], m[i]);
48
49         if (res.second == -1)
50             return {0, -1};
51
52         r = res.first;
53         mod = res.second;
54     }
55
56     return {r, mod};
57 }
58
59 void solve(){
60     vector<ll>a(2);
61     vector<ll>m(2);
62     repl(i,0,2){
63         cin >> a[i] >> m[i];
64     }
65     pair<ll,ll> ans = CRT(a, m);
66     if(ans.second == -1){
67         cout << "-1\n";
68     }
69     else{
70         cout << ans.first << " " << ans.second << "\n";
71     }
72 }

```

1.5. Extended Euclides

```

1 //Encuentra
2 //valores para x e y
3 //A*x + B*y = gcd(A, B)
4 ll extgcd(ll a, ll b, ll &x, ll &y){
5     if(b == 0){
6         x = 1;
7         y = 0;
8         return a;
9     }
10
11     ll x1, y1;
12     ll g = extgcd(b, a % b, x1, y1);
13     x = y1;
14     y = x1 - (a / b) * y1;
15     return g;
16 }
17
18 void solve(){
19     ll A, B, C;
20     cin >> A >> B >> C;
21     ll x, y;
22     ll g = extgcd(A, B, x, y);
23     if(C % g != 0){
24         cout << "-1\n";
25         return;
26     }
27     ll k = -C / g;
28     x *= k;
29     y *= k;
30     cout << x << " " << y << "\n";
31 }

```

1.6. Fibonacci Log(n)

```
1 long long int fibo(int n) {
2     long long int a, b, x, y, tmp;
3     a = x = tmp = 0;
4     b = y = 1;
5     while (n != 0)
6         if (n % 2 == 0) {
7             tmp = a * a + b * b;
8             b = b * a + a * b + b * b;
9             a = tmp;
10            n = n / 2;
11        } else {
12            tmp = a * x + b * y;
13            y = b * x + a * y + b * y;
14            x = tmp;
15            n = n - 1;
16        }
17    // x = fibo(n-1)
18    return y; // y = fibo(n)
19 }
```

1.7. Least Common Multiple

```
1 ll lcm(ll a, ll b){
2     return (b/__gcd(a,b)) * a;
3 }
```

1.8. Modular Inverse

```
1 const long long N = 1e6 + 10;
2 const int MOD = 1e9+7;
3 long long fact[N];
4 long long invfact[N];
5 void init(){
6     fact[0] = 1;
7     for(long long i = 1; i < N; i++)
8         fact[i] = fact[i-1] * i % MOD;
9
10    invfact[N-1] = pote(fact[N-1], MOD-2);
11
12    for(long long i = N-2; i >= 0; i--)
13        invfact[i] = invfact[i+1] * (i+1) % MOD;
14 }
15
16 long long nCk(long long n, long long k){
17     if(k < 0 || k > n) return 0;
18     return fact[n] * invfact[k] % MOD * invfact[n-k] % MOD;
19 }
```

1.9. Pollard Rho Rabin Miller

```
1 map<ll, ll> fact;
2
3 ll pote(ll a, ll b, ll mod) {
4     ll ans = 1;
5     while (b) {
6         if (b & 1) ans = (__int128)ans * a % mod;
7         a = (__int128)a * a % mod;
8         b >>= 1;
9     }
10    return ans;
11 }
12
13 ll mcd(ll a, ll b) {
14     a = abs(a);
15     b = abs(b);
16     while (a != 0) {
17         ll r = b % a;
18         b = a;
19         a = r;
20     }
21    return b;
22 }
```

```
22 }
23
24 //Rabin Miller
25 bool primo(ll n) {
26     if (n < 2) return false;
27     if (n == 2) return true;
28     ll D = n - 1, S = 0;
29     while (D % 2 == 0) {
30         D /= 2;
31         S++;
32     }
33     static const ll STEPS = 1611;
34     for(ll pasos = 0; pasos < STEPS; pasos++){
35         const ll A = 1 + rand() % (n - 1);
36         ll M = pote(A, D, n);
37         if (M == 1 || M == (n - 1)) goto next;
38         for(ll k = 0; k < S-1; k++){
39             M = (__int128)M * M % n;
40             if (M == (n - 1)) goto next;
41         }
42         return false;
43     next:;
44     }
45     return true;
46 }
47
48 //Rho de pollard
49 ll factor(ll n) {
50     static ll A, B;
51     A = 1 + rand() % (n - 1);
52     B = 1 + rand() % (n - 1);
53     #define fun(x) (((__int128)(x) * ((x) + B)) % n + A)
54
55     ll x = 2, y = 2, d = 1;
56     while (d == 1 || d == -1) {
57         x = fun(x);
58         y = fun(fun(y));
59         d = mcd(x - y, n);
60     }
61     return abs(d);
62 }
63
64 void factorize(ll n){
65     assert(n > 0);
66     while (n > 1 && !primo(n)) {
67         ll fx;
68         do {
69             fx = factor(n);
70         } while (fx == n);
71
72         n /= fx;
73         factorize(fx);
74
75         for (auto &it : fact) {
76             while (n % it.first == 0) {
77                 n /= it.first;
78                 it.second++;
79             }
80         }
81     }
82     if (n > 1) fact[n]++;
83 }
```

2. Geometry

2.1. Convex Hull

```
1 // Devuelve sentido horario
2 vector<point> hull(vector<point> p)
3 {
4     int n = p.size();
5     vector<point> h;
6     sort(p.begin(), p.end());
7     for(int i = 0; i < n; i++){
8         while(h.size() >= 2 and p[i].left(h[h.size() - 2], h.back()) < 0){
9             h.pop_back();
10        }
11        h.push_back(p[i]);
12    }
13    h.pop_back();
14    int k = h.size();
```

```

15     for(int i = n-1; i >= 0; i--)
16     {
17         while(h.size() >= k + 2 and p[i].left(h[h.size() - 2], h.back()) < 0){
18             h.pop_back();
19         }
20         h.pb(p[i]);
21     }
22     h.pop_back();
23     return h;
24 }

```

2.2. Point

```

1  struct point
2  {
3      int x, y;
4      point() {}
5      point(int x, int y): x(x), y(y) {}
6      point operator -(point p) {return point(x - p.x, y - p.y);}
7      point operator +(point p) {return point(x + p.x, y + p.y);}
8      int sq() const {return x * x + y * y;}
9      double abs() const {return sqrt(sq());}
10     int operator ^(point p) {return x * p.y - y * p.x;}
11     int operator *(point p) {return x * p.x + y * p.y;}
12     point operator *(int a) {return point(x * a, y * a);}
13     bool operator <(const point& p) const {return x == p.x ? y < p.y : x < p.x;}
14     int left(point a, point b) {
15         return ((b - a) ^ (*this - a)); // 0, -, +
16     }
17 };
18 ostream& operator<<(ostream& os, const point& p) {
19     return os << "(" << p.x << ", " << p.y << ")\n";
20 }
21
22
23 void polarSort(vector<point>& v) {
24     sort(v.begin(), v.end(), [] (point a, point b) {
25         const point origin{0, 0};
26         bool ba = a < origin, bb = b < origin;
27         if (ba != bb) { return ba < bb; }
28         return (a^b) > 0;
29     });
30 }

```

2.3. Closest Pair of Points Lineal

```

1  struct pt {
2      ll x, y;
3      bool operator == (const pt & o) const {
4          return x == o.x and y == o.y;
5      }
6  };
7
8  struct CustomHashPoint {
9      size_t operator()(const pt & p) const {
10         static
11         const uint64_t C =
12             chrono::steady_clock::now().time_since_epoch().count();
13         uint64_t x = p.x;
14         uint64_t y = p.y;
15         x ^= x >> 30;
16         x *= 0xbf58476d1ce4e5b9;
17         x ^= x >> 27;
18         x *= 0x94d049bb133111eb;
19         x ^= x >> 31;
20         y ^= y >> 30;
21         y *= 0xbf58476d1ce4e5b9;
22         y ^= y >> 27;
23         y *= 0x94d049bb133111eb;
24         y ^= y >> 31;
25         return C ^ (x + (y << 1));
26     }
27 };
28
29 ll dist2(pt a, pt b) {
30     ll x = a.x - b.x;
31     ll y = a.y - b.y;
32     return x * x + y * y;
33 }
34
35 pair < int, int > closest_pair(vector < pt > p) {

```

```

36     int n = p.size();
37     unordered_map < pt, int, CustomHashPoint > mp;
38     // puntos repetidos
39     for (int i = 0; i < n; i++) {
40         if (mp.count(p[i]))
41             return {
42                 mp[p[i]],
43                 i
44             };
45         mp[p[i]] = i;
46     }
47     mt19937 rng(
48         chrono::steady_clock::now().time_since_epoch().count()
49     );
50     auto rnd = [&]() {
51         return uniform_int_distribution < int > (0, n - 1)(rng);
52     };
53     ll d2 = dist2(p[0], p[1]);
54     pair < int, int > ans = {
55         0,
56         1
57     };
58     auto upd = [&](int i, int j) {
59         ll nd = dist2(p[i], p[j]);
60         if (nd < d2) {
61             d2 = nd;
62             ans = {
63                 i,
64                 j
65             };
66         }
67     };
68     // conseguir una distancia inicial razonable
69     for (int it = 0; it < 20; it++) {
70         int i = rnd();
71         int j = rnd();
72         while (i == j) j = rnd();
73         upd(i, j);
74     }
75     ll d = sqrtl((long double) d2) + 1;
76     unordered_map < pt, vector < int >, CustomHashPoint > grid;
77     grid.reserve(2 * n);
78     for (int i = 0; i < n; i++) {
79         grid[
80             p[i].x / d,
81             p[i].y / d
82         ].push_back(i);
83     }
84     for (auto & [c, v]: grid) {
85         // misma celda
86         for (int i = 0; i < (int) v.size(); i++) {
87             for (int j = i + 1; j < (int) v.size(); j++) {
88                 upd(v[i], v[j]);
89             }
90         }
91         // celdas vecinas
92         for (int dx = -1; dx <= 1; dx++) {
93             for (int dy = -1; dy <= 1; dy++) {
94                 if (dx == 0 and dy == 0) continue;
95                 pt to = {
96                     c.x + dx,
97                     c.y + dy
98                 };
99                 auto it = grid.find(to);
100                 if (it == grid.end()) continue;
101                 for (int i: v) {
102                     for (int j: it->second) {
103                         upd(i, j);
104                     }
105                 }
106             }
107         }
108     }
109     return ans;
110 }

```

2.4. Closest Pair of Points Nlogn

```

1  struct pt {
2      ll x, y;
3      int id;
4  };
5
6  ll dist2(pt a, pt b) {

```

```

7     ll x = a.x - b.x;
8     ll y = a.y - b.y;
9     return x * x + y * y;
10 }
11
12 pair < int, int > closest_pair(vector < pt > p) {
13     int n = p.size();
14     sort(p.begin(), p.end(), [](pt a, pt b) {
15         if (a.x != b.x) return a.x < b.x;
16         return a.y < b.y;
17     });
18     set < pair < ll, int >> s; // {y, id}
19     ll d2 = dist2(p[0], p[1]);
20     pair < int, int > ans = {
21         p[0].id,
22         p[1].id
23     };
24     int l = 0;
25     for (int i = 0; i < n; i++) {
26         ll d = sqrtl((long double) d2) + 1;
27         // sacar puntos que ya est n demasiado lejos en x
28         while (l < i and p[i].x - p[l].x >= d) {
29             s.erase({
30                 p[l].y,
31                 p[l].id
32             });
33             l++;
34         }
35         // buscar puntos con y cerca
36         auto it = s.lower_bound({
37             p[i].y - d,
38             -1
39         });
40         while (it != s.end() and it -> first <= p[i].y + d) {
41             int j = it -> second;
42             ll nd = dist2(p[i], p[j]);
43             if (nd < d2) {
44                 d2 = nd;
45                 ans = {
46                     p[i].id,
47                     p[j].id
48                 };
49                 d = sqrtl((long double) d2) + 1;
50             }
51             it++;
52         }
53         s.insert({
54             p[i].y,
55             p[i].id
56         });
57     }
58     return ans;
59 }

```

3. Strings

3.1. KMP

```

1 vector<int> kmp(string s){
2     int n = s.size();
3     vector<int> pi(n);
4     for(int i = 1; i < n; i++){
5         int j = pi[i-1];
6         while(j > 0 and s[i] != s[j]){
7             j = pi[j-1];
8         }
9         if(s[i] == s[j]){
10             j++;
11         }
12         pi[i] = j;
13     }
14     return pi;
15 }
16 //Pattern Matching
17 vector<int> ocurrencia(string texto, string patron) {
18     vector<int> P = kmp(patron);
19     int k = 0;
20     vector<int> M;
21     repl(i,0,texto.size()) {
22         while (k > 0 and patron[k] != texto[i]) k = P[k - 1];
23         if (patron[k] == texto[i]) k++;

```

```

24         if (k == patron.size()) {
25             M.pb(i - k + 1);
26             k = P[k - 1];
27         }
28     }
29     return M;
30 }
31
32 //Automata
33 void genAut(string &s) {
34     int n = s.size();
35
36     vector<vector<int>>> aut(n+1, vector<int>(26));
37     vector<int> pi = kmp(s);
38
39     for(int i = 0 ; i < n; i++){
40         aut[i][s[i]-'a'] = i+1;
41     }
42
43     for(int i = 0; i < n+1; i++){
44         for(int c = 0; c < 26; c++){
45             if (aut[i][c]) continue;
46             if (i < n and s[i] == 'a' + c) aut[i][c] = i+1;
47             else if (i > 0) aut[i][c] = aut[pi[i-1]][c];
48             else aut[i][c] = 0;
49         }
50     }
51 }

```

3.2. Z Function

```

1  vector<int> z_function(string s){
2      int n = s.size();
3      vector<int> z(n,0);
4      int l = 0;
5      int r = 0;
6      repl(i,l,n){
7          if(i < r){
8              z[i] = min(r - i, z[i - l]);
9          }
10         while(i + z[i] < n and s[z[i]] == s[i + z[i]]){
11             z[i]++;
12         }
13         if(i + z[i] > r){
14             l = i;
15             r = i + z[i];
16         }
17     }
18     return z;
19 }

```

3.3. Trie

```

1  const int E = 26;
2
3  struct AC {
4      int nodes;
5      vector<array<int, E>>go;
6      vector<bool>terminal;
7
8      AC(){
9          add_node();
10         nodes = 1;
11     }
12
13     void add_node(){
14         terminal.emplace_back();
15         go.emplace_back();
16     }
17
18     void insert(string &s){
19         int pos = 0;
20         for(char ch : s){
21             int c = ch - '0';
22             if(go[pos][c] == 0){
23                 go[pos][c] = nodes++;
24                 add_node();
25             }
26             pos = go[pos][c];
27         }
28         terminal[pos] = true;

```

```

29     }
30
31 };

```

3.4. Suffix Array

```

1  vector<int> suffix_array(string s) {
2      s += char(0);
3      const int n = s.size();
4      vector<int> a(n);
5      iota(a.begin(), a.end(), 0);
6      sort(a.begin(), a.end(), [&] (int i, int j){
7          return s[i] < s[j];
8      });
9      vector<int> mapping(n);
10     mapping[a[0]] = 0;
11     for(int i = 1; i < n; i++){
12         mapping[a[i]] = mapping[a[i-1]] + (s[a[i-1]] != s[a[i]]);
13     }
14     int len = 1;
15     vector<int> sbs(n);
16     vector<int> head(n);
17     vector<int> new_mapping(n);
18     while(len < n){
19         for(int i = 0; i < n; i++) sbs[i] = (a[i] + n - len) % n;
20         for(int i = n-1; i >= 0; i--) head[mapping[a[i]]] = i;
21         for(int i = 0; i < n; i++){
22             int x = sbs[i];
23             a[head[mapping[x]]++] = x;
24         }
25         new_mapping[a[0]] = 0;
26         for(int i = 1; i < n; i++){
27             if(mapping[a[i-1]] != mapping[a[i]]){
28                 new_mapping[a[i]] = new_mapping[a[i-1]] + 1;
29             }
30             else{
31                 int pre = mapping[(a[i-1] + len) % n];
32                 int cur = mapping[(a[i] + len) % n];
33                 new_mapping[a[i]] = new_mapping[a[i-1]] + (pre != cur);
34             }
35         }
36         len <<= 1;
37         swap(mapping, new_mapping);
38     }
39     return vector<int>(a.begin()+1, a.end()); //ignorar a[0] que es el centinela
40 }
41
42 vector<int> lcp_array(vector<int> &a, string s){
43     const int n = s.size();
44     vector<int> rank(n);
45     for(int i = 0; i < n; i++) rank[a[i]] = i;
46     int k = 0;
47     vector<int> lcp(n);
48     for(int i = 0; i < n; i++){
49         if(rank[i] == n-1){
50             k = 0;
51             continue;
52         }
53         int j = a[rank[i]+1];
54         while(i+k < n and j+k < n and s[i+k] == s[j+k]) k++;
55         lcp[rank[i]] = k;
56         if(k) k--;
57     }
58     lcp.pop_back();
59     return lcp;
60 }
61
62 void solve(){
63     //Numero de distintos substr en un string n*(n+1)/2 - sumatoria del LCP
64     //verificar si un string es substring de otro con BS en SA
65     //encontrar el longest common substring en 2 strings (ESTA IMPLEMENTACION)
66     string s, t;
67     cin >> s >> t;
68     if(s.size() > t.size()){
69         swap(s, t);
70     }
71     int n = s.size();
72     int m = t.size();
73     string st = s + '#' + t;
74     vector<int> sa = suffix_array(st);
75     vector<int> lcp = lcp_array(sa, st);
76     // print(sa);
77     // print(lcp);
78     vector<int> tr(n+m);

```

```

79     repl(i,0,n+m){
80         if(sa[i] >=n and sa[i+1]>=n){
81             tr[i] = 0;
82             continue;
83         }
84         if(sa[i] < n and sa[i+1] < n){
85             tr[i] = 0;
86             continue;
87         }
88         int mn = min(sa[i], sa[i+1]);
89         int tam1 = mn + lcp[i];
90         // dbg(tam1);
91         tr[i] = lcp[i];
92         if(tam1 > n){
93             dbg(tam1);
94             tr[i] = lcp[i] - (tam1-n);
95         }
96     }
97     // print(tr);
98     int mx = *max_element(tr.begin(), tr.end());
99     repl(i,0,n+m){
100         // dbg(tr[i]);
101         if(tr[i] == mx){
102             string ans = st.substr(sa[i], mx);
103             cout << ans << '\n';
104             return;
105         }
106     }
107 }
108 //Sabrossus

```

3.5. Aho Corasick

```

1 //AhoCorasick para saber si un string s existe en un texto T
2 const int E = 26;
3 const int N = 1e6+10;
4 int cur;
5 int nodoTerminal[N]; //se guarda el nodo terminal del string i
6 set<int>st; //se guarda que nodos son terminales que aparecen en el texto T
7
8 struct AC {
9     int nodes;
10    vector<int>suf, super;
11    vector<array<int, E>>go;
12    vector<bool>terminal;
13    AC(){
14        add_node();
15        nodes = 1;
16    }
17    void add_node(){
18        terminal.emplace_back();
19        go.emplace_back();
20        super.emplace_back();
21        suf.emplace_back();
22    }
23
24    void insert(string &s){
25        int pos = 0;
26        for(char ch : s){
27            int c = ch - 'a';
28            if(go[pos][c] == 0){
29                go[pos][c] = nodes++;
30                add_node();
31            }
32            pos = go[pos][c];
33        }
34        terminal[pos] = true;
35        nodoTerminal[cur] = pos;
36    }
37
38    void build(){
39        queue<int>Q;
40        Q.emplace(0);
41        while(!Q.empty()){
42            int u = Q.front();
43            Q.pop();
44            super[u] = (suf[u] == 0 or terminal[suf[u]] ? suf[u] : super[suf[u]]);
45            for(int c = 0; c < E; c++){
46                if(go[u][c]){
47                    int v = go[u][c];
48                    Q.emplace(v);
49                    suf[v] = u == 0 ? 0 : go[suf[u]][c];
50                }

```

```

51         else{
52             go[u][c] = u == 0 ? 0 : go[suf[u]][c];
53         }
54     }
55 }
56 }
57 };
58
59 void mark(int u, AC &ac){
60     if(u == 0) return;
61     mark(ac.super[u], ac);
62     if(ac.terminal[u]){
63         st.insert(u);
64         ac.terminal[u] = false;
65     }
66     ac.super[u] = 0;
67 }
68
69 void solve(){
70     int n;
71     cin >> n;
72     AC ac;
73     repl(i,0,n){
74         string s;
75         cin >> s;
76         cur = i;
77         ac.insert(s);
78     }
79     string t; //texto grande
80     cin >> t;
81     ac.build();
82     int p = 0;
83     for(auto x : t){
84         int c = x - 'a';
85         p = ac.go[p][c];
86         mark(p, ac);
87     }
88     repl(i,0,n){
89         if(st.count(nodoTerminal[i])){
90             cout << "YES\n";
91         }
92         else{
93             cout << "NO\n";
94         }
95     }
96 }
97
98 //-----
99
100 //AhoCorasick para Frecuencias
101 const int E = 26;
102 const int N = 1e6+10;
103 int nodoTerminal[N];
104 int cur;
105 struct AC {
106     int nodes;
107     vector<int> suf, freq, by_level;
108     vector<bool> terminal;
109     vector<array<int,E>> go;
110
111     AC(){
112         add_node();
113         nodes = 1;
114     }
115     void add_node(){
116         terminal.emplace_back();
117         suf.emplace_back();
118         go.emplace_back();
119         freq.emplace_back();
120     }
121
122     void insert(string &s){
123         int pos = 0;
124         for(char ch:s){
125             int c = ch - 'a';
126             if(go[pos][c] == 0){
127                 go[pos][c] = nodes++;
128                 add_node();
129             }
130             pos = go[pos][c];
131         }
132         terminal[pos] = true;
133         nodoTerminal[cur] = pos;
134     }
135
136     void build(){

```

```

137     queue<int>Q;
138     Q.emplace(0);
139     while(!Q.empty()){
140         int u = Q.front();
141         Q.pop();
142         by_level.emplace_back(u);
143         for(int c = 0; c < E; c++){
144             if(go[u][c]){
145                 int v = go[u][c];
146                 Q.emplace(v);
147                 suf[v] = u == 0 ? 0 : go[suf[u]][c];
148             }
149             else{
150                 go[u][c] = u == 0 ? 0 : go[suf[u]][c];
151             }
152         }
153     }
154 }
155 };
156
157 void solve(){
158     int n;
159     cin >> n;
160     AC ac;
161     repl(i,0,n){
162         string s;
163         cin >> s;
164         cur = i;
165         ac.insert(s);
166     }
167     string t;
168     cin >> t;
169     ac.build();
170     int p = 0;
171     for(char ch:t){
172         int c = ch-'a';
173         p = ac.go[p][c];
174         ac.freq[p]++;
175     }
176     for(int i = (int)ac.by_level.size()-1; i > 0; i--){
177         int x = ac.by_level[i];
178         ac.freq[ac.suf[x]] += ac.freq[x];
179     }
180     repl(i,0,n){
181         int termi = nodoTerminal[i];
182         cout << ac.freq[termi] << '\n';
183     }
184 }

```

3.6. Booths Algorithm

```

1 //Algoritmo para menor rotacion lexicografica de un string en O(N)
2
3 int menorRotacion(string &s){
4     int n = s.size();
5     vector<int>f(2*n, -1);
6     int k = 0; //indice mejor rotacion
7     for(int j = 1; j < 2*n; j++){
8         int i = f[j-k-1];
9         //el operador %n simula la concatenacion de la cadena s+s
10        while(i!=-1 and s[j%n] != s[(k+i+1)%n]){
11            if(s[j%n] < s[(k+i+1)%n]){
12                k = j - i - 1;
13            }
14            i = f[i];
15        }
16        if(i == -1 and s[j%n] != s[(k+i+1)%n]){
17            if(s[j % n] < s[(k+i+1)%n]){
18                k = j;
19            }
20            f[j - k] = -1;
21        }
22        else{
23            f[j - k] = i + 1;
24        }
25    }
26    return k;
27 }
28 void solve(){
29     string s;
30     cin >> s;
31     ll n = s.size();
32     ll ans = menorRotacion(s);
33     repl(i, ans, ans+n){

```

```

34         cout << s[i%n];
35     }
36 }

```

3.7. Hashing 2D

```

1  /*
2   2D Hashing: buscar todas las apariciones de una matriz
3   r x c dentro de una matriz n x m.
4   */
5
6   const ll BASE = 911382323LL;
7
8   void solve() {
9       int r, c;
10      cin >> r >> c;
11
12      vector<string> pattern(r);
13      for (int i = 0; i < r; i++)
14          cin >> pattern[i];
15
16      // Potencias para el rolling vertical.
17      vector<ll> power(r + 1, 1);
18      for (int i = 1; i <= r; i++)
19          power[i] = power[i - 1] * BASE;
20
21      // Hash de la matriz patr n.
22      ll patternHash = 0;
23
24      for (int i = 0; i < r; i++) {
25          Hash h(pattern[i]);
26          ll rowHash = h.get(0, c);
27
28          patternHash += rowHash * power[r - i - 1];
29      }
30
31      int n, m;
32      cin >> n >> m;
33
34      vector<string> grid(n);
35      for (int i = 0; i < n; i++)
36          cin >> grid[i];
37
38      // rowHash[i][j] = hash de grid[i][j ... j+c-1].
39      vector<vector<ll>> rowHash(
40          n, vector<ll>(m - c + 1)
41      );
42
43      for (int i = 0; i < n; i++) {
44          Hash h(grid[i]);
45
46          for (int j = 0; j <= m - c; j++)
47              rowHash[i][j] = h.get(j, j + c);
48      }
49
50      // matrixHash[i][j] = hash de la submatriz r x c desde (i, j).
51      vector<vector<ll>> matrixHash(
52          n - r + 1,
53          vector<ll>(m - c + 1)
54      );
55
56      for (int j = 0; j <= m - c; j++) {
57          ll cur = 0;
58
59          // Primera ventana vertical.
60          for (int i = 0; i < r; i++)
61              cur += rowHash[i][j] * power[r - i - 1];
62
63          matrixHash[0][j] = cur;
64
65          // Rolling vertical.
66          for (int i = 1; i <= n - r; i++) {
67              cur -= rowHash[i - 1][j] * power[r - 1];
68              cur = cur * BASE + rowHash[i + r - 1][j];
69
70              matrixHash[i][j] = cur;
71          }
72      }
73
74      // 2D difference array para marcar las apariciones.
75      vector<vector<int>> diff(
76          n + 1, vector<int>(m + 1, 0)
77      );
78

```

```

79     for (int i = 0; i <= n - r; i++) {
80         for (int j = 0; j <= m - c; j++) {
81             if (matrixHash[i][j] == patternHash) {
82                 diff[i][j]++;
83                 diff[i][j + c]--;
84                 diff[i + r][j]--;
85                 diff[i + r][j + c]++;
86             }
87         }
88     }
89 }
90
91 // Prefijos 2D.
92 for (int i = 0; i < n; i++)
93     for (int j = 1; j < m; j++)
94         diff[i][j] += diff[i][j - 1];
95
96 for (int j = 0; j < m; j++)
97     for (int i = 1; i < n; i++)
98         diff[i][j] += diff[i - 1][j];
99
100 // Imprimir celdas pertenecientes a alguna aparici n.
101 for (int i = 0; i < n; i++) {
102     for (int j = 0; j < m; j++) {
103         cout << (diff[i][j] > 0 ? grid[i][j] : '.');
104     }
105     cout << '\n';
106 }
107 }

```

3.8. Hashing

```

1  struct Hash {
2      long long P=1777771,MOD[2],PI[2];
3      vector<long long> h[2],pi[2];
4
5      // SOLO PARA ROLLING / JOIN:
6      vector<long long> pw[2];
7
8      Hash(string& s){
9          MOD[0]=999727999;
10         MOD[1]=107077777;
11         PI[0]=325255434;
12         PI[1]=10018302;
13
14         for(int k = 0; k < 2; k++) {
15             h[k].resize(s.size()+1);
16             pi[k].resize(s.size()+1);
17
18             // SOLO PARA ROLLING / JOIN:
19             pw[k].resize(s.size()+1);
20         }
21
22         for(int k = 0; k < 2; k++){
23             h[k][0] = 0;
24             pi[k][0] = 1;
25
26             // SOLO PARA ROLLING / JOIN:
27             pw[k][0] = 1;
28
29             long long p = 1;
30
31             for(int i = 1; i < s.size()+1; i++){
32                 h[k][i] = (h[k][i-1]+p*s[i-1])%MOD[k];
33                 pi[k][i] = (1LL*pi[k][i-1]*PI[k])%MOD[k];
34
35                 // SOLO PARA ROLLING / JOIN:
36                 pw[k][i] = (1LL*pw[k][i-1]*P)%MOD[k];
37
38                 p = (p*P)%MOD[k];
39             }
40         }
41     }
42
43     long long get(int s, int e){
44         long long h0 = (h[0][e]-h[0][s]+MOD[0])%MOD[0];
45         h0 = (1LL*h0*pi[0][s])%MOD[0];
46
47         long long h1 = (h[1][e]-h[1][s]+MOD[1])%MOD[1];
48         h1 = (1LL*h1*pi[1][s])%MOD[1];
49
50         return (h0<<32)|h1;
51     }
52 }

```

```
53 // SOLO PARA ROLLING / JOIN:
54 long long join(long long a, long long b, int lenA){
55     long long a0 = a>>32;
56     long long a1 = a&0xffffffffLL;
57
58     long long b0 = b>>32;
59     long long b1 = b&0xffffffffLL;
60
61     a0 = (a0 + b0*pw[0][lenA])%MOD[0];
62     a1 = (a1 + b1*pw[1][lenA])%MOD[1];
63
64     return (a0<<32)|a1;
65 }
66 };
```

3.9. Manacher

```
1  const int N = 1e5 + 10;
2  int odd[N]; //odd[i] = max odd palindrome centered on i
3  int even[N]; //even[i] = max even palindrome centered on i
4  void manacher(string & s) {
5      int l = 0, r = -1, n = s.size();
6      for (int i = 0; i < n; i++) {
7          int k = i > r ? 1 : min(odd[l + r - i], r - i);
8          while (i + k < n and i - k >= 0 and s[i + k] == s[i - k]) k++;
9          odd[i] = k--;
10         if (i + k > r) l = i - k, r = i + k;
11     }
12     l = 0;
13     r = -1;
14     for (int i = 0; i < n; i++) {
15         int k = i > r ? 0 : min(even[l + r - i + 1], r - i + 1);
16         k++;
17         while (i + k <= n and i - k >= 0 and s[i + k - 1] == s[i - k]) k++;
18         even[i] = --k;
19         if (i + k - 1 > r) l = i - k, r = i + k - 1;
20     }
21 }
```

4. Bit Manipulation

4.1. Binary Representation

```
1  string tobit(long long x) {
2      string a(64, '0');
3      for (int i = 63; i >= 0; --i) {
4          if (x & 1) a[i] = '1';
5          x >>= 1;
6      }
7      return a;
8  }
```

4.2. Bit Mask

```
1  for(int mask = 0; mask < (1 << n); mask++) { //(1 << n) = 0(2^n)
2      for(int i = 0; i < n; i++) { // 0(n)
3          //verificamos si el i-esimo bit esta encendido
4          if(mask & (1 << i)) {
5              cout << v[i] << " ";
6          }
7      }
8  }
```

4.3. Bits Prendidos En Un Numero X En Una Posicion Pos

```
1  //retorna el numero de bits prendidos
2  //en una posicion pos desde 0 hasta el numero num
3  //tambien se puede usar en rangos asi:
4  //prendidos(R, 1) - prendidos(L-1, 1);
5  ll prendidos(ll num, ll pos){
6      if(num <= 0){
7          return 0;
```

```
8     }
9     ll pote = (1ll << pos);
10    ll blocks = (num + 1) / pote;
11    ll ones = blocks / 2;
12    ones *= pote;
13    if(blocks & 1){
14        ones += (num + 1) % pote;
15    }
16    return ones;
17 }
18 //Sabrosson
```

4.4. Turn Off a Bit at Position X

```
1 void solve(){
2     //mapa de caracteres en windows para ~
3     int a, x;
4     cin >> a >> x; // a es el numero, x es el numero de bit
5     //los bits se enumeran de dercha a izquierda iniciando de 0
6     /*
7     Ej: n = 8
8     Binario:      1 0 0 0
9     Posicion:     3 2 1 0
10    */
11    a = a & ~(1 << x); //apagamos el bit x de a
12    cout << a << "\n";
13 }
```

4.5. Xor Basis

```
1 ll base[LOG];
2 ll tam = 0;
3 void add(ll msk){
4     for(int i = LOG-1; i>= 0; i--){
5         if(!(msk&(1LL<<i)))continue;
6         if(!base[i]){
7             base[i]=msk;
8             tam++;
9             return;
10        }
11        msk^=base[i];
12    }
13 }
```

5. Data Structures

5.1. Segment Tree

```
1 const int N = 1e5;
2
3 struct info {
4     int num;
5 };
6
7 info ST[4 * N];
8
9 info ope(info u, info v) {
10     return {u.num + v.num};
11 }
12
13 void build(const vector<int> &v, int in = 1, int L = 0, int R = n - 1) {
14     if (L == R) {
15         ST[in] = {v[L]}; // Asignamos el valor desde el vector
16     } else {
17         int mid = (L + R) >> 1;
18         build(v, 2 * in, L, mid);
19         build(v, (2 * in) + 1, mid + 1, R);
20         ST[in] = ope(ST[2 * in], ST[(2 * in) + 1]);
21     }
22 }
23
24 void update(int pos, int val, int in = 1, int L = 0, int R = n - 1) {
25     if (L == R) {
26         ST[in].num = val;
27     } else {
```

```

28     int mid = (L + R) >> 1;
29     if (pos <= mid) {
30         update(pos, val, 2 * in, L, mid);
31     } else {
32         update(pos, val, (2 * in) + 1, mid + 1, R);
33     }
34     ST[in] = ope(ST[2 * in], ST[(2 * in) + 1]);
35 }
36 }
37
38 info get(int l, int r, int in = 1, int L = 0, int R = n - 1) {
39     if (r < L or R < l) return {0}; // Elemento neutro
40     if (l <= L and R <= r) return ST[in];
41
42     int mid = (L + R) >> 1;
43     info left = get(l, r, 2 * in, L, mid);
44     info right = get(l, r, (2 * in) + 1, mid + 1, R);
45     return ope(left, right);
46 }
47
48 // Llamadas simples y limpias:
49 // build(v); // Construye con el vector
50 // update(2, 10); // Cambia el valor en pos 2 a 10
51 // info ans = get(1, 3); // Obtiene la respuesta en el rango [1, 3]
52 // v = {1 2 3 4 5}
53 // get(1, 0, n-1, 1, 3) = 2 + 3 + 4 = 9

```

5.2. Segment Tree Lazy Propagation

```

1  const int N = 1e5;
2
3  struct info {
4      long long num;
5  };
6
7  info ST[4 * N];
8  long long lazy[4 * N];
9
10 info ope(info u, info v) {
11     return {u.num + v.num};
12 }
13
14 void build(const vector<int> &v, int in = 1, int L = 0, int R = n - 1) {
15     if (L == R) {
16         ST[in] = {v[L]};
17     } else {
18         int mid = (L + R) >> 1;
19
20         build(v, 2 * in, L, mid);
21         build(v, 2 * in + 1, mid + 1, R);
22
23         ST[in] = ope(ST[2 * in], ST[2 * in + 1]);
24     }
25 }
26
27 void push(int in, int L, int R) {
28     if (lazy[in] == 0)
29         return;
30
31     ST[in].num += lazy[in] * (R - L + 1);
32
33     if (L != R) {
34         lazy[2 * in] += lazy[in];
35         lazy[2 * in + 1] += lazy[in];
36     }
37
38     lazy[in] = 0;
39 }
40
41 void update(int l, int r, long long val,
42             int in = 1, int L = 0, int R = n - 1) {
43
44     push(in, L, R);
45
46     if (r < L or R < l)
47         return;
48
49     if (l <= L and R <= r) {
50         lazy[in] += val;
51         push(in, L, R);
52         return;
53     }
54

```

```

55     int mid = (L + R) >> 1;
56
57     update(l, r, val, 2 * in, L, mid);
58     update(l, r, val, 2 * in + 1, mid + 1, R);
59
60     ST[in] = ope(ST[2 * in], ST[2 * in + 1]);
61 }
62
63 info get(int l, int r,
64         int in = 1, int L = 0, int R = n - 1) {
65
66     push(in, L, R);
67
68     if (r < L or R < l)
69         return {0};
70
71     if (l <= L and R <= r)
72         return ST[in];
73
74     int mid = (L + R) >> 1;
75
76     info left = get(l, r, 2 * in, L, mid);
77     info right = get(l, r, 2 * in + 1, mid + 1, R);
78
79     return ope(left, right);
80 }

```

5.3. Bit 1D

```

1  struct BIT{
2      int n;
3      vector<ll> bit;
4      BIT(int _n){
5          n = _n;
6          bit.assign(n + 1, 0);
7      }
8      void add(int x, ll val){
9          for (int i = x; i <= n; i += (i&(-i))){
10             bit[i] += val;
11         }
12     }
13
14     ll sum(int x){
15         ll ans = 0ll;
16         for (int i = x; i > 0; i -= (i&(-i))){
17             ans += bit[i];
18         }
19         return ans;
20     }
21     ll query(int l, int r){
22         return sum(r) - sum(l - 1);
23     }
24 };
25 // Para actualizar en rango y consultas puntuales
26 // bit.add(l, val); bit.add(r+1, val);
27 // bit.query(l, pos)

```

5.4. Bit 2D

```

1  struct BIT{
2      int n, m;
3      vector<vector<ll>> bit;
4
5      BIT(int _n){
6          n = _n;
7          m = _n;
8          bit.assign(n+1, vector<ll>(n+1, 0));
9      }
10     void add(int x, int y, ll val){
11         for (int i = x; i <= n; i += (i&(-i))){
12             for (int j = y; j <= m; j += (j&(-j))){
13                 bit[i][j] += val;
14             }
15         }
16     }
17     ll sum(int x, int y){
18         ll ans = 0ll;
19         for (int i = x; i > 0; i -= (i&(-i))){
20             for (int j = y; j > 0; j -= (j&(-j))){
21                 ans += bit[i][j];
22             }
23         }
24     }
25 }

```

```

23     }
24     return ans;
25 }
26 ll query(int x1, int y1, int x2, int y2){
27     return sum(x2, y2) - sum(x2, y1 - 1) - sum(x1 - 1, y2) + sum(x1 - 1, y1 - 1);
28 }
29 };

```

5.5. Bit 2D Infinite Plane

```

1  struct BIT{
2      ll n;
3      vector<vector<ll>> xs;
4      vector<vector<ll>> bit;
5      BIT(ll nn){
6          n = nn;
7          xs.resize(n + 1);
8          bit.resize(n + 1);
9      }
10     void prepare(ll x, ll y){
11         for (ll i = x; i <= n; i += (i&(-i))){
12             xs[i].push_back(y);
13         }
14     }
15     void build(){
16         for (ll i = 1; i <= n; i++){
17             sort(xs[i].begin(), xs[i].end());
18             xs[i].erase(unique(xs[i].begin(), xs[i].end()), xs[i].end());
19             bit[i].assign(xs[i].size() + 1, 0);
20         }
21     }
22     void add(ll x, ll y, ll val){
23         for (ll i = x; i <= n; i += (i&(-i))){
24             ll p = lower_bound(xs[i].begin(), xs[i].end(), y) - xs[i].begin() + 1;
25             for (ll j = p; j < bit[i].size(); j += (j&(-j))){
26                 bit[i][j] += val;
27             }
28         }
29     }
30     ll sum(ll x, ll y){
31         ll ans = 0;
32         for (ll i = x; i > 0; i -= (i&(-i))){
33             ll p = lower_bound(xs[i].begin(), xs[i].end(), y) - xs[i].begin();
34             for (ll j = p; j > 0; j -= (j&(-j))){
35                 ans += bit[i][j];
36             }
37         }
38         return ans;
39     }
40     ll query(ll x1, ll y1, ll x2, ll y2){
41         return sum(x2-1,y2) - sum(x2-1, y1) - sum(x1 - 1, y2) + sum(x1 - 1, y1);
42     }
43 };
44
45 void solve(int tc){
46     int n,q; cin >> n >> q;
47     set<ll> X;
48     set<ll> Y;
49     vector<vector<ll>> points;
50     vector<vector<ll>> query;
51     repl(i,0,n){
52         ll x,y,w; cin >> x >> y >> w;
53         X.insert(x);
54         Y.insert(y);
55         points.push_back({x,y,w});
56     }
57     repl(i,0,q){
58         int t; cin >> t;
59         if(t == 0){
60             ll x,y,w; cin >> x >> y >> w;
61             X.insert(x);
62             Y.insert(y);
63             query.push_back({0,x,y,w});
64         }
65         else{
66             ll x1, y1, x2, y2;
67             cin >> x1 >> y1 >> x2 >> y2;
68             X.insert(x1);
69             X.insert(x2);
70             Y.insert(y1);
71             Y.insert(y2);
72             query.push_back({1,x1,y1,x2,y2});
73         }
74     }

```

```

75     map<ll,ll> equis, ye;
76     int i = 0;
77     for(ll x : X){
78         i++;
79         equis[x] = i;
80     }
81     i = 0;
82     for(ll x : Y){
83         i++;
84         ye[x] = i;
85     }
86     BIT bit((ll)X.size());
87     for(auto &x : points){
88         x[0] = equis[x[0]];
89         x[1] = ye[x[1]];
90         bit.prepare(x[0], x[1]);
91     }
92     for(auto &x : query){
93         if(x[0] == 0){
94             x[1] = equis[x[1]];
95             x[2] = ye[x[2]];
96             bit.prepare(x[1], x[2]);
97         }
98         else{
99             x[1] = equis[x[1]];
100            x[2] = ye[x[2]];
101            x[3] = equis[x[3]];
102            x[4] = ye[x[4]];
103        }
104    }
105    bit.build();
106    for(auto &x : points){
107        bit.add(x[0], x[1], x[2]);
108    }
109    for(auto &x : query){
110        if(x[0] == 0){
111            bit.add(x[1], x[2], x[3]);
112        }
113        else{
114            cout << bit.query(x[1],x[2],x[3],x[4]) << "\n";
115        }
116    }
117 }

```

5.6. Bit 3D

```

1  struct BIT {
2      int n;
3      int m;
4      int p;
5      vector < vector < vector < ll >>> bit;
6      BIT(int _n) {
7          n = _n + 10;
8          m = _n + 10;
9          p = _n + 10;
10         bit.assign(n, vector < vector < ll >> (n, vector < ll > (n, 0)));
11     }
12     void add(int x, int y, int z, int val) {
13         for (int i = x; i <= n; i += (i & (-i))) {
14             for (int j = y; j <= n; j += (j & (-j))) {
15                 for (int k = z; k <= n; k += (k & (-k))) {
16                     bit[i][j][k] += val;
17                 }
18             }
19         }
20     }
21     ll sum(int x, int y, int z) {
22         ll ans = 0 ll;
23         for (int i = x; i > 0; i -= (i & (-i))) {
24             for (int j = y; j > 0; j -= (j & (-j))) {
25                 for (int k = z; k > 0; k -= (k & (-k))) {
26                     ans += bit[i][j][k];
27                 }
28             }
29         }
30         return ans;
31     }
32     ll query(int x1, int y1, int z1, int x2, int y2, int z2) {
33         return sum(x2, y2, z2) - sum(x1 - 1, y2, z2) - sum(x2, y2, z1 - 1) - sum(x2, y1 - 1, z2)
34             + sum(x1 - 1, y1 - 1, z2) + sum(x2, y1 - 1, z1 - 1) + sum(x1 - 1, y2, z1 - 1) -
35             sum(x1 - 1, y1 - 1, z1 - 1);
36     }
37 };

```

5.7. DSU Dynamic Representative

```

1  int n, q;
2  cin >> n >> q;
3  DSU dsu(n + q + 10);
4  vector<int> id(n + 1), box(n + q + 10);
5  int nodes = n;
6  iota(id.begin(), id.end(), 0);
7  iota(box.begin(), box.end(), 0);
8  for(int i = 0 ; i < q ; i++){
9      int a, b;
10     cin >> a >> b;
11     int ori = id[a];
12     int ori2 = id[b];
13     dsu.parent[ori] = ori2;
14     nodes++;
15     id[a] = nodes;
16     box[nodes] = a;
17 }
18 for(int i = 1; i <= n ; i++){
19     cout << box[dsu.find(i)] << ' ';
20 }

```

5.8. Disjoint Set Union

```

1  struct DSU{
2      int num;
3      vector<int> parent;
4      vector<int> sizes;
5      DSU(){};
6      DSU(int n){
7          init(n);
8      }
9      void init(int n){
10         num = n;
11         parent.resize(n+1);
12         sizes.resize(n+1);
13         //si tienes index 0 inicializar desde 0
14         for(int i = 1; i <= n ; i++){
15             parent[i] = i;
16             sizes[i] = 1;
17         }
18     }
19     int find(int a){
20         if(a == parent[a]) return a;
21         return parent[a] = find(parent[a]);
22     }
23     void join(int a, int b){
24         int x = find(a);
25         int y = find(b);
26         if(x == y) return;
27         num--;
28         if(sizes[x] < sizes[y]){
29             parent[x] = y;
30             sizes[y] += sizes[x];
31         }
32         else{
33             parent[y] = x;
34             sizes[x] += sizes[y];
35         }
36     }
37 }
38 };
39 //Sabrossus

```

5.9. Dynamic Connectivity Offline

```

1  struct Change {
2      int a, b, sz_a;
3  };
4
5  struct DSU {
6      vector<int> parent, sz;
7      vector<Change> history;
8      int components;
9
10     DSU(int n) {
11         parent.resize(n + 1);
12         sz.assign(n + 1, 1);
13         iota(parent.begin(), parent.end(), 0);

```

```

14     components = n;
15 }
16
17 int find(int x) {
18     while (x != parent[x])
19         x = parent[x];
20     return x;
21 }
22
23 bool unite(int a, int b) {
24     a = find(a);
25     b = find(b);
26     if (a == b) {
27         history.push_back({-1, -1, -1});
28         return false;
29     }
30     if (sz[a] < sz[b])
31         swap(a, b);
32     history.push_back({a, b, sz[a]});
33     parent[b] = a;
34     sz[a] += sz[b];
35     components--;
36     return true;
37 }
38
39 int snapshot() {
40     return history.size();
41 }
42
43 void rollback(int snap) {
44     while (history.size() > snap) {
45         auto [a, b, old_sz_a] = history.back();
46         history.pop_back();
47         if (a == -1)
48             continue;
49         parent[b] = b;
50         sz[a] = old_sz_a;
51         components++;
52     }
53 }
54 };
55
56 struct Edge {
57     int u, v;
58 };
59
60 struct SegmentTree {
61     int n;
62     vector<vector<Edge>> tree;
63     SegmentTree(int n) {
64         this->n = n;
65         tree.resize(4 * n + 5);
66     }
67     void add(int node, int l, int r, int ql, int qr, Edge e) {
68         if (qr < l or r < ql)
69             return;
70         if (ql <= l and r <= qr) {
71             tree[node].push_back(e);
72             return;
73         }
74         int mid = (l + r) / 2;
75         add(node * 2, l, mid, ql, qr, e);
76         add(node * 2 + 1, mid + 1, r, ql, qr, e);
77     }
78     void add(int l, int r, Edge e) {
79         if (l > r)
80             return;
81         add(1, 1, n, l, r, e);
82     }
83 };
84
85 void dfs(int node, int l, int r, SegmentTree &seg, DSU &dsu, vector<int> &answer){
86     int snap = dsu.snapshot();
87     for (auto [u, v] : seg.tree[node])
88         dsu.unite(u, v);
89     if (l == r) {
90         answer[l] = dsu.components;
91     } else {
92         int mid = (l + r) / 2;
93         dfs(node * 2, l, mid, seg, dsu, answer);
94         dfs(node * 2 + 1, mid + 1, r, seg, dsu, answer);
95     }
96     dsu.rollback(snap);
97 }
98
99 void solve() {

```

```

100     int n, q;
101     cin >> n >> q;
102     if (q == 0)
103         return;
104     DSU dsu(n);
105     SegmentTree seg(q);
106     map<pair<int,int>, int> start;
107     vector<int> answer(q + 1);
108     vector<bool> query(q + 1);
109     for (int t = 1; t <= q; t++) {
110         char type;
111         cin >> type;
112         if (type == '?') {
113             query[t] = true;
114         } else {
115             int u, v;
116             cin >> u >> v;
117             if (u > v)
118                 swap(u, v);
119             if (type == '+') {
120                 start[{u, v}] = t;
121             } else {
122                 int L = start[{u, v}];
123                 seg.add(L, t - 1, {u, v});
124                 start.erase({u, v});
125             }
126         }
127     }
128     for (auto [edge, L] : start) {
129         auto [u, v] = edge;
130         seg.add(L, q, {u, v});
131     }
132     dfs(1, 1, q, seg, dsu, answer);
133     for (int t = 1; t <= q; t++) {
134         if (query[t])
135             cout << answer[t] << '\n';
136     }
137 }

```

5.10. Implicit Treap

```

1  // Mantiene las posiciones
2  struct Node {
3      int val , pri , sz ;
4      Node *l , *r;
5      Node ( int v ) {
6          val = v;
7          pri = rand () ;
8          sz = 1;
9          l = r = nullptr ;
10     }
11 };
12 using pnode = Node *;
13 int sz ( pnode t ) {
14     return t ? t -> sz : 0;
15 }
16 void upd (pnode t) {
17     if (t) t -> sz = 1 + sz (t -> l) + sz (t -> r);
18 }
19 void split ( pnode t , pnode &l , pnode &r , int k ) {
20     if (!t) {
21         l = r = nullptr ;
22         return ;
23     }
24     int leftSize = sz (t -> l);
25     if ( leftSize >= k ) {
26         split (t -> l , l , t -> l , k) ;
27         r = t;
28     }
29     else {
30         split (t -> r , t -> r , r , k - leftSize - 1) ;
31         l = t;
32     }
33     upd (t);
34 }
35 pnode merge ( pnode l , pnode r ) {
36     if (!l || !r ) return l ? l : r;
37
38     if (l -> pri > r -> pri ) {
39         l -> r = merge (l -> r , r);
40         upd (l );
41         return l;
42     }
43     else {

```

```

44     r -> l = merge (l , r ->l );
45     upd (r );
46     return r;
47 }
48 }
49 void insert ( pnode & root , int pos , int val ) {
50     pnode a , b;
51     split ( root , a , b , pos );
52     root = merge ( merge (a , new Node ( val )) , b ) ;
53 }
54 void erase ( pnode & root , int pos ) {
55     pnode a , b , c;
56     split ( root , a , b , pos );
57     split (b , b , c , 1) ;
58     root = merge (a , c) ;
59 }
60 void print(pnode t) {
61     if (!t) return;
62
63     print(t->l);
64     cout << t->val << " ";
65     print(t->r);
66 }
67 int get(pnode t, int k) {
68     if (!t) return -1;
69
70     int leftSize = sz(t->l);
71
72     if (k < leftSize)
73         return get(t->l, k);
74
75     if (k == leftSize)
76         return t->val;
77
78     return get(t->r, k - leftSize - 1);
79 }

```

5.11. Ordered Set

```

1  #include <bits/stdc++.h>
2  #include <ext/pb_ds/assoc_container.hpp>
3  #include <ext/pb_ds/tree_policy.hpp>
4
5  using namespace std;
6  using namespace __gnu_pbds;
7
8  typedef tree<int, null_type, less<int>, rb_tree_tag, tree_order_statistics_node_update>
9      ordered_set;
10
11 void solve() {
12     ordered_set st;
13
14     st.insert(10);
15     st.insert(30);
16     st.insert(20);
17
18     // 1. Elemento en la posicion K (0-indexed, de menor a mayor)
19     // find_by_order devuelve un iterador (puntero). Usamos '*' para el valor.
20     auto it = st.find_by_order(1);
21     if (it != st.end()) {
22         int elemento = *it; // Devuelve 20 de forma segura
23     }
24
25     // 2. Cuantos elementos son estrictamente menores que X
26     int menores_que_25 = st.order_of_key(25); // Devuelve 2 (el 10 y el 20)
27 }
28 // Sabrossus

```

5.12. Range Bit

```

1  struct RangeBIT{
2      int n;
3      vector<ll> bit1, bit2;
4      RangeBIT(int nn){
5          n = nn;
6          bit1.assign(n + 1,0);
7          bit2.assign(n + 1,0);
8      }
9      void add_one(vector<ll> &b, int x, ll val){
10         for (int i = x; i <= n; i+= (i&(-i))){
11             b[i] += val;

```

```

12     }
13 }
14 void add(int l, int r, ll val){
15     add_one(bit1, l, val);
16     add_one(bit1, r + 1, -val);
17     add_one(bit2, l, l * val);
18     add_one(bit2, r + 1, -(r + 1) * val);
19 }
20 ll sum_one(const vector<ll> &b, int x){
21     ll ans = 0ll;
22     for (int i = x; i > 0; i -= (i&(-i))){
23         ans += b[i];
24     }
25     return ans;
26 }
27
28 ll sum(int x){
29     return (x + 1) * sum_one(bit1,x) - sum_one(bit2, x);
30 }
31 ll query(int l, int r){
32     return sum(r) - sum(l - 1);
33 }
34 };

```

5.13. Sparse Table

```

1  const int N = 1e5;
2  const int LOG = 20;
3  int v[N]; //values
4  int st[N][LOG]; //Sparse Table
5  int lg[N]; // log values
6  int n;
7  void build(){
8      for(int i = 0; i < n; i++) st[i][0] = v[i];
9
10     for(int i = 1; i < LOG; i++){
11         for(int j = 0; j < n - (1<<i) + 1; j++){
12             st[j][i] = min(st[j][i-1], st[j+(1<<(i-1))][i-1]);
13         }
14     }
15     lg[1] = 0;
16     for(int i = 2; i <= n; i++) lg[i] = lg[i/2] + 1;
17 }
18 int query(int L, int R){
19     int k = lg[R-L+1];
20     return min(st[L][k], st[R-(1<<k)+1][k]);
21 }
22 // v = [5, 2, 4, 7, 1, 3]
23 // query(1, 4)      [2, 4, 7, 1]

```

5.14. Treap

```

1  // Ordena los elementos por valor
2  struct Node {
3      int key, pri, sz;
4      Node *l, *r;
5      Node(int k) {
6          key = k;
7          pri = rand();
8          sz = 1;
9          l = r = nullptr;
10     }
11 };
12
13 using pnode = Node*;
14
15 int sz(pnode t) {
16     return t ? t->sz : 0;
17 }
18
19 void upd(pnode t) {
20     if (t) t->sz = 1 + sz(t->l) + sz(t->r);
21 }
22
23 void split(pnode t, pnode &l, pnode &r, int k) {
24     if (!t) {
25         l = r = nullptr;
26         return;
27     }
28
29     int leftSize = sz(t->l);

```

```
30
31     if (leftSize >= k) {
32         split(t->l, l, t->l, k);
33         r = t;
34     } else {
35         split(t->r, t->r, r, k - leftSize - 1);
36         l = t;
37     }
38
39     upd(t);
40 }
41
42 pnode merge(pnode l, pnode r) {
43     if (!l || !r) return l ? l : r;
44
45     if (l->pri > r->pri) {
46         l->r = merge(l->r, r);
47         upd(l);
48         return l;
49     } else {
50         r->l = merge(l, r->l);
51         upd(r);
52         return r;
53     }
54 }
55
56 void insert(pnode &root, int pos, int val) {
57     pnode a, b;
58     split(root, a, b, pos);
59     root = merge(merge(a, new Node(val)), b);
60 }
61
62 void erase(pnode &root, int pos) {
63     pnode a, b, c;
64     split(root, a, b, pos);
65     split(b, b, c, 1);
66     root = merge(a, c);
67 }
68
69 int get(pnode t, int k) {
70     if (!t) return -1;
71
72     int leftSize = sz(t->l);
73
74     if (k < leftSize)
75         return get(t->l, k);
76
77     if (k == leftSize)
78         return t->key;
79
80     return get(t->r, k - leftSize - 1);
81 }
82
83 void print(pnode t) {
84     if (!t) return;
85
86     print(t->l);
87     cout << t->key << " ";
88     print(t->r);
89 }
```

6. Dynamic Programing

6.1. Bounded Knapsack

```
1 // bounded_knapsack para DPs del estilo :
2 // dp[j - cw] + cv , w-> pesos-> valor y quieres maximizar el valor
3 int n, we;
4 cin >> n >> we;
5 vector<int> v, m, w, dp;
6 v.resize(n);
7 m.resize(n);
8 w.resize(n);
9 dp.assign(we+1, 0);
10 int sum = 0;
11 for(int i = 0 ; i < n ; i++){
12     int a, b, c;
13     cin >> a >> b >> c;
14     v[i] = a;
15     w[i] = b;
16     m[i] = c;
```

```

17     vector<int> old = dp;
18     for(int r = 0; r < w[i] and r <= we ; r++){
19         deque<pair<int,int>> dq;
20         for(int j = r; j <= we ; j += w[i]){
21             int k = (j - r)/w[i];
22             while(!dq.empty() and dq.front().first < k - m[i]){
23                 dq.pop_front();
24             }
25             int cur = old[j] - k * v[i];
26             while(!dq.empty() && dq.back().second <= cur){
27                 dq.pop_back();
28             }
29             dq.push_back({k, cur});
30             dp[j] = dq.front().second + k * v[i];
31         }
32     }
33 }
34 cout << dp[we];

```

6.2. Kadane 2D

```

1  int n;
2  cin >> n;
3  vector < vector < int >> v(n);
4  repl(i, 0, n) {
5      repl(j, 0, n) {
6          int x;
7          cin >> x;
8          v[i].push_back(x);
9      }
10 }
11 int maxi = INT_MIN;
12 repl(ini, 0, n) {
13     vector < int > pref(n, 0);
14     repl(i, ini, n) {
15         repl(j, 0, n) {
16             pref[j] += v[i][j];
17         }
18         int cur = 0;
19         repl(j, 0, n) {
20             cur = max(pref[j], pref[j] + cur);
21             maxi = max(maxi, cur);
22         }
23     }
24 }

```

6.3. Kadane 3D

```

1  int n;
2  cin >> n;
3
4  repl(i, 0, n) {
5      repl(j, 0, n) {
6          repl(k, 0, n) {
7              cin >> v[i][j][k];
8          }
9      }
10 }
11 int maxi = INT_MIN;
12 repl(k_ini, 0, n) {
13     vector < vector < int >> pref2(n, vector < int > (n, 0));
14     repl(k, k_ini, n) {
15         repl(i, 0, n) {
16             repl(j, 0, n) {
17                 pref2[i][j] += v[i][j][k];
18             }
19         }
20         repl(ini, 0, n) {
21             vector < int > pref(n, 0);
22             repl(i, ini, n) {
23                 repl(j, 0, n) {
24                     pref[j] += pref2[i][j];
25                 }
26                 int cur = 0;
27                 repl(u, 0, n) {
28                     cur = max(pref[u], cur + pref[u]);
29                     maxi = max(maxi, cur);
30                 }
31             }
32         }
33     }

```

```

34     }
35 }
36
37 cout << maxi << "\n";

```

6.4. SOS DP

```

1 void sos_dp(vector<ll>& dp, int K){
2 // k es el tamaño del arreglo :v
3     for(int bit = 0; bit < K; bit++){
4         for(int mask = 0; mask < (1 << K); mask++){
5             if(mask & (1 << bit)){
6                 dp[mask] += dp[mask ^ (1 << bit)];
7 //                 si quieres que sea un sos dp de maximos
8 //                 dp[mask] = max(dp[mask], dp[mask ^ (1 << bit)]);
9             }
10        }
11    }
12 }

```

6.5. Fastfibonacci

```

1 #include <bits/stdc++.h>
2 #define ll long long
3
4 using namespace std;
5
6 const ll MOD = 1e9 + 7;
7
8 struct Matrix {
9     ll a[2][2];
10    Matrix() {
11        memset(a, 0, sizeof(a));
12    }
13 };
14
15 Matrix multiply(Matrix A, Matrix B) {
16     Matrix C;
17     for (int i = 0; i < 2; i++) {
18         for (int j = 0; j < 2; j++) {
19             for (int k = 0; k < 2; k++) {
20                 C.a[i][j] = (C.a[i][j] + A.a[i][k] * B.a[k][j]) % MOD;
21             }
22         }
23     }
24     return C;
25 }
26
27 Matrix power(Matrix A, ll n) {
28     Matrix R;
29     R.a[0][0] = 1;
30     R.a[1][1] = 1;
31     while (n) {
32         if (n & 1)
33             R = multiply(R, A);
34         A = multiply(A, A);
35         n >>= 1;
36     }
37     return R;
38 }
39
40 ll fibonacci(ll n) {
41     if (n == 0) return 0;
42     Matrix A;
43     A.a[0][0] = 1;
44     A.a[0][1] = 1;
45     A.a[1][0] = 1;
46     A.a[1][1] = 0;
47     Matrix R = power(A, n - 1);
48     return R.a[0][0];
49 }
50
51 int main() {
52     ll n;
53     cin >> n;
54     cout << fibonacci(n) << "\n";
55 }

```

7. Flow

7.1. Dinic

```

1  struct Dinic
2  {
3      const int INF = 1e9;
4
5      struct flowEdge
6      {
7          int to, rev, f, cap;
8      };
9
10     vector<vector<flowEdge>> G;
11     vector<int> work, lvl, Q;
12
13     int S, T, nodes;
14
15     Dinic(){}
16     Dinic(int n, int s, int t){
17         S = s;
18         T = t;
19         nodes = n;
20         G.clear();
21         G.resize(n);
22         Q.resize(n);
23     }
24     void addEdge(int st, int en, int cap){
25         flowEdge A = {en, (int)G[en].size(), 0, cap};
26         flowEdge B = {st, (int)G[st].size(), 0, 0};
27         G[st].pb(A);
28         G[en].pb(B);
29     }
30     int dfs(int v, int f){
31         if(v == T or f == 0) return f;
32         for(int &i = work[v]; i < G[v].size(); i++){
33             flowEdge &e = G[v][i];
34             int u = e.to;
35             if(e.cap <= e.f or lvl[u] != lvl[v] + 1) continue;
36             int df = dfs(u, min(f, e.cap - e.f));
37             if(df){
38                 e.f += df;
39                 G[u][e.rev].f -= df;
40                 return df;
41             }
42         }
43         return 0;
44     }
45     bool bfs(){
46         int qt = 0;
47         Q[qt++] = S;
48         lvl.assign(nodes, -1);
49         lvl[S] = 0;
50         for(int i = 0; i < qt; i++){
51             int u = Q[i];
52             for(flowEdge &e : G[u]){
53                 int v = e.to;
54                 if(e.cap <= e.f or lvl[v] != -1) continue;
55                 lvl[v] = lvl[u] + 1;
56                 Q[qt++] = v;
57             }
58         }
59         return lvl[T] != -1;
60     }
61     int maxFlow(){
62         int flow = 0;
63         while(bfs()){
64             work.assign(nodes, 0);
65             while(true){
66                 int df = dfs(S, INF);
67                 if(df == 0) break;
68                 flow += df;
69             }
70         }
71         return flow;
72     }
73 }
74 };

```

8. Graph Algorithms

8.1. Dijkstra

```

1  const int N = 1e5;
2
3  int vis[N];
4  vector<pair<int,int>> G[N];
5
6  void dijkstra(int x){
7      priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
8      pq.push({0,x});
9      vis[x] = 0;
10     while(!pq.empty()){
11         auto y = pq.top();
12         pq.pop();
13         int u = y.second;
14         int cost = y.first;
15         for(auto v : G[u]){
16             int cur = cost + v.second;
17             if(cur < vis[v.first]){
18                 vis[v.first] = cur;
19                 pq.push({cur,v.first});
20             }
21         }
22     }
23 }

```

8.2. Kruskal MST

```

1  const int N = 1e5;
2  struct DSU{
3      int num;
4      vector<int> parent;
5      vector<int> sizes;
6      DSU(){};
7      DSU(int n){
8          init(n);
9      }
10     void init(int n){
11         num = n;
12         parent.resize(n+1);
13         sizes.resize(n+1);
14         for(int i = 1; i <= n ; i++){
15             parent[i] = i;
16             sizes[i] = 1;
17         }
18     }
19     int find(int a){
20         if(a == parent[a]) return a;
21         return parent[a] = find(parent[a]);
22     }
23     void join(int a, int b){
24         int x = find(a);
25         int y = find(b);
26         if(x == y) return;
27         num--;
28         if(sizes[x] < sizes[y]){
29             parent[x] = y;
30             sizes[y] += sizes[x];
31         }
32         else{
33             parent[y] = x;
34             sizes[x] += sizes[y];
35         }
36     }
37 };
38 int n;
39 vector<pair<int,pair<int,int>>> List;
40 int Kruskal_MST(){
41     sort(List.begin(),List.end());
42     DSU dsu(n);
43     int mst = 0, t = 1;
44     for(int i = 0; i < List.size(); i++){
45         int w = List[i].first; //weight
46         int a = List[i].second.first; //at
47         int b = List[i].second.second; //to
48         if(dsu.find(a) != dsu.find(b)){
49             dsu.join(a,b);
50             mst += w;
51             t++;
52         }
53         if(t == n) break;
54     }

```

```

55     return mst;
56 }

```

8.3. Topological Sort Kahn Algorithm

```

1  const int N = 1e5;
2
3  int indegree[N];
4  vector<int> G[N];
5  int n;
6  vector<int> Kahn(){
7      queue<int> q;
8      vector<int> ans;
9      for(int i = 0; i < n; i++){
10         if(indegree[i] == 0) q.push(i);
11     }
12     while (!q.empty()){
13         int v = q.front();
14         q.pop();
15         ans.push_back(v);
16         for(int u : G[v]){
17             indegree[u]--;
18             if(indegree[u] == 0){
19                 q.push(u);
20             }
21         }
22     }
23     //if(ans.size() != n) grafo no DAG
24     return ans;
25 }

```

8.4. Kosaraju

```

1  struct Kosaraju {
2      int n, components;
3      vector<vector<int>> graph, invgraph;
4      vector<int> vis, order, component;
5
6      Kosaraju(int n) : n(n) {
7          graph.resize(n + 1);
8          invgraph.resize(n + 1);
9          vis.resize(n + 1);
10         component.resize(n + 1);
11     }
12
13     void add_edge(int u, int v) {
14         graph[u].push_back(v);
15         invgraph[v].push_back(u);
16     }
17
18     void dfs1(int u) {
19         vis[u] = 1;
20         for (int v : graph[u]) {
21             if (!vis[v])
22                 dfs1(v);
23         }
24         order.push_back(u);
25     }
26
27     void dfs2(int u, int id) {
28         vis[u] = 1;
29         component[u] = id;
30         for (int v : invgraph[u]) {
31             if (!vis[v])
32                 dfs2(v, id);
33         }
34     }
35     void build() {
36         for (int u = 1; u <= n; u++) {
37             if (!vis[u])
38                 dfs1(u);
39         }
40         reverse(order.begin(), order.end());
41         fill(vis.begin(), vis.end(), 0);
42         components = 0;
43         for (int u : order) {
44             if (!vis[u]) {
45                 components++;
46                 dfs2(u, components);
47             }
48         }
49     }
50 }

```

```

49     }
50 };

```

8.5. Kuhn

```

1  int n;
2  vector<vector<int>> G;
3  vector<int> mt;
4  vector<bool> used;
5  bool try_kuhn(int v){
6      if(used[v]) return false;
7      used[v] = true;
8      for(int to : G[v]){
9          if(mt[to] == -1 or try_kuhn(mt[to])){
10             mt[to] = v;
11             return true;
12         }
13     }
14     return false;
15 }
16 void solve(int tc){
17     mt.assign(n, -1);
18     repl(v,0,n){
19         used.assign(n, false);
20         try_kuhn(v);
21     }
22 }

```

8.6. 2SAT

```

1  struct TwoSAT {
2      int n, nodes;
3      vector<vector<int>> graph, invgraph;
4      vector<int> vis, order, component;
5      vector<bool> ans;
6
7      TwoSAT(int n) : n(n), nodes(2 * n) {
8          graph.resize(nodes);
9          invgraph.resize(nodes);
10         vis.resize(nodes);
11         component.resize(nodes);
12         ans.resize(n);
13     }
14
15     int id(int x, bool value) {
16         return 2 * x + value;
17     }
18
19     int neg(int x) {
20         return x ^ 1;
21     }
22
23     void add_implication(int a, int b) {
24         graph[a].push_back(b);
25         invgraph[b].push_back(a);
26     }
27
28     void add_or(int a, int b) {
29         add_implication(neg(a), b);
30         add_implication(neg(b), a);
31     }
32
33     void add_xor(int a, int b) {
34         add_or(a, b);
35         add_or(neg(a), neg(b));
36     }
37
38     void add_equivalence(int a, int b) {
39         add_implication(a, b);
40         add_implication(b, a);
41         add_implication(neg(a), neg(b));
42         add_implication(neg(b), neg(a));
43     }
44
45     void dfs1(int u) {
46         vis[u] = 1;
47         for (int v : graph[u]) {
48             if (!vis[v])
49                 dfs1(v);
50         }
51         order.push_back(u);

```

```
52     }
53
54     void dfs2(int u, int id) {
55         vis[u] = 1;
56         component[u] = id;
57         for (int v : invgraph[u]) {
58             if (!vis[v])
59                 dfs2(v, id);
60         }
61     }
62
63     bool solve() {
64         for (int u = 0; u < nodes; u++) {
65             if (!vis[u])
66                 dfs1(u);
67         }
68         reverse(order.begin(), order.end());
69         fill(vis.begin(), vis.end(), 0);
70         int cnt = 0;
71         for (int u : order) {
72             if (!vis[u]) {
73                 dfs2(u, cnt++);
74             }
75         }
76         for (int i = 0; i < n; i++) {
77             if (component[2 * i] != component[2 * i + 1])
78                 return false;
79             ans[i] = component[2 * i] > component[2 * i + 1];
80         }
81         return true;
82     }
83 };
```

9. Tree Algorithms

9.1. Euler Tour

```
1  const int N = 1e5 + 10;
2  vector<int> path;
3  vector<int> G[N];
4  void EulerTour(int node, int parent){
5      path.pb(node);
6      for(int u : G[node]){
7          if(u != parent){
8              EulerTour(u,node);
9          }
10     }
11     path.pb(node);
12 }
13
14 const int N = 1e5 + 10;
15 int tin[N];
16 int tout[N];
17 int flat[N];
18 int timer = 1;
19 void EulerTour(int node, int parent){
20     tin[node] = timer;
21     flat[timer] = node;
22     timer++;
23     for(int u : G[node]){
24         if(u != parent){
25             EulerTour(u, node);
26         }
27     }
28     tout[node] = timer;
29 }
```

9.2. Binary Lifting

```
1  const int N = 1e5;
2  const int LOG = 20;
3  int up[N][LOG];
4  vector<int> G[N];
5  //Find parents
6  void dfs(int u){
7      for(int v : G[u]){
8          up[v][0] = u;
9          for(int j = 1; j < LOG; j++){
```

```

10         up[v][j] = up[up[v][j-1]][j-1];
11     }
12     dfs(v);
13 }
14 }
15 int get_K_parent(int a, int k){
16     for(int j = LOG-1; j >= 0; j++){
17         if(k & (1<<j)){
18             a = up[a][j];
19         }
20     }
21     return a;
22 }

```

9.3. Centroid Decomposition

```

1  //EJERCICIO IMPLEMENTACION XENIA AND TREE
2  //Nodo rojo mas cercano a un nodo v
3
4  const int N = 2e5+10;
5  vector<int> G[N];
6  int tag[N]; // time of discovery of centroid
7  int fat[N]; // father in centroid decomposition
8  int szt[N]; // size of subtree
9
10 int best[N];
11 int niv[N];
12 int dist[30][N]; //dist[nivel][nodo];
13
14 void dfs_level(int node, int p, int niv, int prof){
15     dist[niv][node] = prof;
16     for(auto y : G[node]){
17         if(y != p and tag[y] < 0){
18             dfs_level(y, node, niv, prof+1);
19         }
20     }
21 }
22 // -----CENTROID-----
23 int calcsz(int node, int p){
24     szt[node] = 1;
25     for(auto y : G[node]){
26         if(y != p and tag[y] < 0){
27             szt[node] += calcsz(y, node);
28         }
29     }
30     return szt[node];
31 }
32 int ccnt = 0;
33 void cdfs(int node, int p, int nivel = 0, int sz = -1){
34     if(sz < 0) sz = calcsz(node, -1);
35     for(auto y : G[node]){
36         if(tag[y] < 0 and szt[y] * 2 >= sz){
37             szt[node] = 0;
38             cdfs(y, p, nivel, sz);
39             return;
40         }
41     }
42     tag[node] = ccnt++;
43     fat[node] = p;
44     niv[node] = nivel;
45     dfs_level(node, -1, nivel, 0);
46     for(auto y : G[node]){
47         if(tag[y] < 0){
48             cdfs(y, node, nivel+1);
49         }
50     }
51 }
52
53 int n;
54 void centroid(){
55     ccnt = 0;
56     repl(1,1,n+1){
57         tag[i] = -1;
58     }
59     cdfs(1, -1);
60 }
61
62 //-----
63
64 void update(int node, int rojo){
65     int nivel = niv[node];
66     int cost = dist[nivel][rojo];
67     best[node] = min(best[node], cost);
68     int p = fat[node];

```

```

69     if(p > 0){
70         update(p, rojo);
71     }
72 }
73
74 int improve(int node, int cur){
75     if(node <= 0){
76         return 1e9;
77     }
78     int nivel = niv[node];
79     int cost = best[node] + dist[nivel][cur];
80     int p = fat[node];
81     return min(cost, improve(p, cur));
82 }
83
84 void solve(){
85     int qq;
86     cin >> n >> qq;
87     repl(i,0,30){
88         repl(j,1,n+1){
89             dist[i][j] = -1;
90             best[j] = 1e9;
91         }
92     }
93     repl(i,0,n-1){
94         int a, b;
95         cin >> a >> b;
96         G[a].push_back(b);
97         G[b].push_back(a);
98     }
99     centroid();
100    update(1, 1);
101    while(qq--){
102        int q, node;
103        cin >> q >> node;
104        if(q == 1){
105            update(node, node);
106        }
107        else{
108            int res = best[node];
109            int res2 = improve(node, node);
110            cout << min(res, res2) << '\n';
111        }
112    }
113 }
114
115 int main(){
116     velocito;
117     int t = 1;
118     // cin >> t;
119     while(t--){
120         solve();
121     }
122     return 0;
123 }
124 //Sabrossus
125 //TC Argentina

```

9.4. HLD

```

1  struct HLD {
2      int n, act;
3      vector<vector<int>> adj;
4      vector<int> parent, depth, heavy, head, pos;
5      vector<int> val;
6      vector<int> seg;
7      int tam;
8      HLD(int n) : n(n) {
9          adj.resize(n+1);
10         parent.resize(n+1);
11         depth.resize(n+1);
12         heavy.assign(n+1, -1);
13         head.resize(n+1);
14         pos.resize(n+1);
15         val.resize(n+1);
16         act = 0;
17     }
18
19     void add_edge(int u, int v) {
20         adj[u].push_back(v);
21         adj[v].push_back(u);
22     }
23
24     int dfs(int u, int p) {

```

```

25         parent[u]=p;
26         int size=1, max_sub=0;
27         for (int v : adj[u]){
28             if (v==p) continue;
29             depth[v]=depth[u]+1;
30             int sub=dfs(v,u);
31             if (sub>max_sub){
32                 max_sub=sub;
33                 heavy[u]=v;
34             }
35             size+=sub;
36         }
37         return size;
38     }
39
40     void decompose(int u, int h) {
41         head[u]=h;
42         pos[u]=act++;
43         if (heavy[u]!=-1) decompose(heavy[u], h);
44         for (int v : adj[u]){
45             if (v!=parent[u] and v!=heavy[u]) decompose(v,v);
46         }
47     }
48
49     void build(){
50         tam=1;
51         while (tam < n) tam<<=1;
52         seg.assign(2*tam,inf);
53         for (int i = 1; i <= n; i++){
54             seg[tam+pos[i]]=val[i];
55         }
56         for (int i = tam - 1; i > 0; i--){
57             seg[i]=max(seg[2*i],seg[2*i+1]);
58         }
59     }
60
61     void update(int u, int value) {
62         int i=tam+pos[u];
63         seg[i]=value;
64         for (i>>=1; i; i>>=1){
65             seg[i]=max(seg[2*i],seg[2*i+1]);
66         }
67     }
68
69     int query(int l, int r) {
70         int res=inf;
71         l+=tam;
72         r+=tam;
73         while (l <= r){
74             if (l&1) res=max(res,seg[l++]);
75             if (!(r&1)) res=max(res,seg[r--]);
76             l>>=1;
77             r>>=1;
78         }
79         return res;
80     }
81
82     void init(){
83         int root=1;
84         dfs(root, 0);
85         decompose(root,root);
86         build();
87     }
88
89     int query_path(int a, int b){
90         int res=inf;
91         while (head[a]!=head[b]){
92             if (depth[head[a]]<depth[head[b]]) swap(a,b);
93             res=max(res,query(pos[head[a]],pos[a]));
94             a=parent[head[a]];
95         }
96         if (depth[a]>depth[b]) swap(a,b);
97         res=max(res,query(pos[a],pos[b]));
98         return res;
99     }
100 };
101
102 void solve(){
103     int n,q;
104     cin>>n>>q;
105     HLD hld(n);
106     for (int i = 1; i <= n; i++){
107         cin>>hld.val[i];
108     }
109     for (int i = 0; i < n-1; i++){
110         int a,b;

```

```

111         cin>>a>>b;
112         hld.add_edge(a,b);
113     }
114     hld.init();
115     while(q--){
116         int t;
117         cin>>t;
118         if(t==1){
119             int s,x;
120             cin>>s>>x;
121             hld.update(s,x);
122         }
123         else{
124             int a,b;
125             cin>>a>>b;
126             cout<<hld.query_path(a,b)<<'␣';
127         }
128     }
129 }

```

9.5. Lowest Common Ancestor

```

1  const int N = 1e5;
2  const int LOG = 20;
3  vector<int> G[N];
4  int depth[N];
5  int up[N][LOG];
6  // Assign levels and get parents
7  void dfs(int u){
8      for(int v : G[u]){
9          depth[v] = depth[u] + 1;
10         up[v][0] = u;
11         for(int j = 1; j < LOG; j++){
12             up[v][j] = up[up[v][j-1]][j-1];
13         }
14         dfs(v);
15     }
16 }
17 int get_lca(int a, int b){
18     if(depth[a] < depth[b]) swap(a,b);
19     int k = depth[a] - depth[b];
20     for(int j = LOG-1; j >= 0; j--){
21         if(k & (1<<j)){
22             a = up[a][j];
23         }
24     }
25     if(a == b) return a;
26     for(int j = LOG-1; j >= 0; j--){
27         if(up[a][j] != up[b][j]){
28             a = up[a][j];
29             b = up[b][j];
30         }
31     }
32     return up[a][0];
33 }

```

10. Utilities

10.1. Prime Numbers

1	2	3	5	7	11	13	17	19	23	29
2	31	37	41	43	47	53	59	61	67	71
3	73	79	83	89	97	101	103	107	109	113
4	127	131	137	139	149	151	157	163	167	173
5	179	181	191	193	197	199	211	223	227	229
6	233	239	241	251	257	263	269	271	277	281
7	283	293	307	311	313	317	331	337	347	349
8	353	359	367	373	379	383	389	397	401	409
9	419	421	431	433	439	443	449	457	461	463
10	467	479	487	491	499	503	509	521	523	541
11	547	557	563	569	571	577	587	593	599	601
12	607	613	617	619	631	641	643	647	653	659
13	661	673	677	683	691	701	709	719	727	733
14	739	743	751	757	761	769	773	787	797	809
15	811	821	823	827	829	839	853	857	859	863
16	877	881	883	887	907	911	919	929	937	941
17	947	953	967	971	977	983	991	997	1009	1013
18	1019	1021	1031	1033	1039	1049	1051	1061	1063	1069

```

19 1087 1091 1093 1097 1103 1109 1117 1123 1129 1151
20 1153 1163 1171 1181 1187 1193 1201 1213 1217 1223
21 1229 1231 1237 1249 1259 1277 1279 1283 1289 1291
22 1297 1301 1303 1307 1319 1321 1327 1361 1367 1373
23 1381 1399 1409 1423 1427 1429 1433 1439 1447 1451
24 1453 1459 1471 1481 1483 1487 1489 1493 1499 1511
25 1523 1531 1543 1549 1553 1559 1567 1571 1579 1583
26 1597 1601 1607 1609 1613 1619 1621 1627 1637 1657
27 1663 1667 1669 1693 1697 1699 1709 1721 1723 1733
28 1741 1747 1753 1759 1777 1783 1787 1789 1801 1811
29 1823 1831 1847 1861 1867 1871 1873 1877 1879 1889
30 1901 1907 1913 1931 1933 1949 1951 1973 1979 1987
31
32          970'997 971'483    921'281'269    999'279'733
33 1'000'000'009 1'000'000'021 1'000'000'409 1'005'012'527
34 10005512850571600911

```

10.2. ASCII TABLE

```

1  ASCII QUICK REFERENCE
2  =====
3
4  CHARACTER RANGES
5  -----
6  '0' ... '9'      48 ... 57      -> c - '0'
7  'A' ... 'Z'      65 ... 90      -> c - 'A'
8  'a' ... 'z'      97 ... 122     -> c - 'a'
9
10 RELATIONSHIPS
11 -----
12 'a' - 'A' = 32
13 'A' + 32 = 'a'
14 'a' - 32 = 'A'
15
16 DIGITS
17 -----
18 '0' = 48      '5' = 53
19 '1' = 49      '6' = 54
20 '2' = 50      '7' = 55
21 '3' = 51      '8' = 56
22 '4' = 52      '9' = 57
23
24 UPPERCASE
25 -----
26 A 65  B 66  C 67  D 68  E 69  F 70  G 71
27 H 72  I 73  J 74  K 75  L 76  M 77  N 78
28 O 79  P 80  Q 81  R 82  S 83  T 84  U 85
29 V 86  W 87  X 88  Y 89  Z 90
30
31 LOWERCASE
32 -----
33 a 97  b 98  c 99  d 100 e 101 f 102 g 103
34 h 104 i 105 j 106 k 107 l 108 m 109 n 110
35 o 111 p 112 q 113 r 114 s 115 t 116 u 117
36 v 118 w 119 x 120 y 121 z 122
37
38 COMMON SYMBOLS
39 -----
40 SPACE 32      ! 33      " 34  # 35  $ 36
41 % 37  & 38  ' 39  ( 40  ) 41
42 * 42  + 43  , 44  - 45  . 46
43 / 47  : 48  ; 49  < 50  = 51
44 > 52  ? 53  @ 54  [ 55  \ 56  ] 57
45 ^ 58  _ 59  ` 60  { 61  | 62  ~ 63
46 } 64  ~ 65
47
48 USEFUL TRICKS
49 -----
50 int x = c - '0'; // digit -> integer
51 char c = '0' + x; // integer -> digit
52
53 int x = c - 'a'; // lowercase -> [0,25]
54 int x = c - 'A'; // uppercase -> [0,25]
55
56 islower(c)
57 isupper(c)
58 isdigit(c)
59 isalpha(c)
60 isalnum(c)
61 isspace(c)
62 tolower(c)
63 toupper(c)
64 =====

```

10.3. Int128

```

1  // --- LECTURA B SICA ---
2  __int128 read128() {
3      string s;
4      cin >> s;
5      __int128 n = 0;
6      int sign = 1;
7      size_t start = 0;
8
9      if (!s.empty() && s[0] == '-') {
10         sign = -1;
11         start = 1;
12     }
13
14     for (size_t i = start; i < s.length(); ++i) {
15         n = n * 10 + (s[i] - '0');
16     }
17
18     return n * sign;
19 }
20
21 // --- IMPRESI N B SICA ---
22 void print128(__int128 n) {
23     if (n == 0) {
24         cout << 0;
25         return;
26     }
27     if (n < 0) {
28         cout << '-';
29         n = -n;
30     }
31     string s;
32     while (n > 0) {
33         s += (char)('0' + (n % 10));
34         n /= 10;
35     }
36     reverse(s.begin(), s.end());
37     cout << s;
38 }

```

10.4. Merge Sort

```

1  // Rango de uso: 0-indexed.
2  // Llamar en el solve como: mergeSort(v, 0, n - 1);
3  ll ans = 0;
4
5  void mergeSort(vector<ll> &v, int low, int high) {
6      if (low == high) return;
7
8      int mid = (low + high) >> 1;
9      mergeSort(v, low, mid);
10     mergeSort(v, mid + 1, high);
11
12     queue<ll> L, H;
13     for (int i = low; i < mid + 1; i++) {
14         L.push(v[i]);
15     }
16     for (int i = mid + 1; i < high + 1; i++) {
17         H.push(v[i]);
18     }
19
20     for (int i = low; i < high + 1; i++) {
21         if (L.size() == 0) {
22             v[i] = H.front();
23             H.pop();
24         }
25         else if (H.size() == 0) {
26             v[i] = L.front();
27             L.pop();
28         }
29         else {
30             if (L.front() <= H.front()) {
31                 v[i] = L.front();
32                 L.pop();
33             }
34             else {
35                 v[i] = H.front();
36                 H.pop();
37                 ans += L.size(); // Inversion detectada
38             }
39         }
40     }
41 }

```

```
41 }
```

10.5. Prefix Sum 2D

```
1 //1 indexed
2 const int N = 1005;
3 int v[N][N];
4 int acu[N][N];
5 int n, m;
6
7 void solve() {
8     //limpiar la acu para cada TC
9
10    // Construcción: acumula 1 si el elemento es exactamente 1
11    for (int i = 1; i <= n; i++) {
12        for (int j = 1; j <= m; j++) {
13            int val = (v[i][j] == 1 ? 1 : 0);
14            acu[i][j] = val + acu[i - 1][j] + acu[i][j - 1] - acu[i - 1][j - 1];
15        }
16    }
17
18    // Esquinas de consulta:
19    // (a, b) -> Superior Izquierda
20    // (c, d) -> Inferior Derecha
21    int a = 2, b = 2, c = 4, d = 4; // Valores de ejemplo
22
23    int ans = acu[c][d] - acu[a - 1][d] - acu[c][b - 1] + acu[a - 1][b - 1];
24    cout << ans << '\n';
25 }
```

10.6. Priority Queue Min Heap

```
1 priority_queue<int, vector<int>, greater<int>> pq;
```

10.7. Random

```
1 mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
2 // mt19937_64 para 64 bits usar uint64_t o unsigned long long
3 cout << rng() << endl;
4 shuffle(v.begin(), v.end(), rng);
```

10.8. Plantilla Python

```
1 import sys
2 from bisect import bisect_left, bisect_right, insort
3 from collections import defaultdict, Counter, deque
4 from heapq import heappush, heappop, heapify
5 from math import gcd, lcm, isqrt
6
7 input = sys.stdin.readline
8
9
10 def solve():
11     n = int(input()) #leer numero
12     a,b,c= map(int, input().split()) #leer numeros
13     s = input().strip() #leer cadenas
14     v = list(map(int, input().split())) #leer lista de enteros
15
16
17 t = 1
18 # t = int(input())
19 for _ in range(t):
20     solve()
21
22 # C++                                Python
23
24 # vector                                list
25 # array                                list
26 # string                                str
27
28 # set                                    set
29 # map                                    dict
30 # multiset                              Counter
31 # unordered_map                         dict
```

```
32 # unordered_set          set
33
34 # priority_queue         heapq
35 # queue                  deque
36 # deque                  deque
37
38 # lower_bound             bisect_left
39 # upper_bound             bisect_right
40 # sort                    list.sort()
41 # binary_search           bisect_left/right
42
43 # pair                    tuple
44 # struct                  class / dataclass
45
46 # gcd                     math.gcd
47 # lcm                     math.lcm
48
49 # push_back               append
50 # push_front              appendleft
51 # pop_back                pop
52 # pop_front               popleft
53
54 # size()                  len()
55 # empty()                 not x
```

10.9. Pragma

```
1 #pragma GCC optimize("O3,unroll-loops")
2 #pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
```